Day 1 · Git and GitHub 101

> A short, friendly primer for someone who has never used Git or GitHub. By the end you will have Git installed, a GitHub account, your identity configured, a project copied to your machine, and you will know how to pull the latest changes down and push your own changes up. Nothing here assumes you have done any of this before.

This guide is deliberately slow and complete. Every new word is explained the first time it appears. Work through it top to bottom, and actually type the commands as you go, because the only way this becomes second nature is by doing it once yourself.

---

1. What are Git and GitHub, and why should you care?

Imagine you are writing a document. You save it, then you make changes, then you save again. Now the earlier version is gone. If today's changes turn out to be a mistake, you cannot go back. And if a teammate is editing the same document, you have no clean way to combine your work with theirs.

Version control solves this. It is a system that records the full history of a project: every change, who made it, when, and why. You can look back at any past version, undo a bad change, and merge your work with other people's work safely.

Git is the most widely used version control tool in the world. It runs on your own computer and keeps that complete history for you. Git is a program you install, like any other program.

GitHub is a website that stores Git projects online, so you can back them up and share them with other people. Think of it this way:

- **Git** is the tool on your machine that tracks changes. It works even with no internet.
- **GitHub** is a place on the internet where a copy of the project lives, so a whole team can work on the same project from different computers.

A rough analogy: Git is like the "track changes" and "version history" feature built into your editor, and GitHub is like a shared drive in the cloud where everyone keeps the master copy. They are made to work together, but they are not the same thing. Git is the tool; GitHub is one of several services that host Git projects (GitLab and Bitbucket are two others).

A few words you will meet constantly, defined once here:

- **Repository** (or **repo**): a single project that Git is tracking. It is just a folder, plus a hidden history of everything that has happened to it.
- **Commit**: a saved snapshot of your project at one moment, with a short message describing what changed. History is a chain of commits.
- **Remote**: a copy of the repository that lives somewhere else, usually on GitHub. The remote is what your teammates share.
- **Clone**: to make a full copy of a remote repository onto your own machine for the first time.

- **Pull**: to bring down the latest changes from the remote into your local copy.
- **Push**: to send your own committed changes up to the remote so others can get them.

That is the whole mental model. The rest of this guide is just doing each of those steps.

---

2. Install Git

Git is a single program. Pick your operating system below, run the install, then verify it.

Windows

1. Go to https://git-scm.com/download/win in your browser. The download starts on its own.
2. Run the installer you just downloaded. When it asks you questions, the default answer on every screen is fine for now. Keep clicking Next, then Install.
3. This also gives you an app called **Git Bash**, a terminal window where you can type Git commands. You will use it for everything below.

macOS

The easiest way is Homebrew, a popular installer for developer tools. Open the **Terminal** app and run:

```
brew install git
```

If you do not have Homebrew, running `git --version` in the Terminal will offer to install Git's command line tools for you. Accept that offer.

Linux (Ubuntu or Debian)

Open a terminal and run:

```
sudo apt update
sudo apt install git
```

(Other Linux families use their own installer, for example `sudo dnf install git` on Fedora.)

Verify the install

Open your terminal (Git Bash on Windows, Terminal on macOS or Linux) and type:

```
git --version
```

You should see a line like `git version 2.46.0`. The exact number does not matter. If you see a version, Git is installed and you are ready.

---

3. Create a GitHub account

This is the online side. It is free.

1. Go to https://github.com and select **Sign up**.
2. Enter your email address, choose a password, and pick a **username**. The username is public and becomes part of your project web addresses, so pick something clean and professional.
3. GitHub will email you a code to confirm your address. Enter it.
4. When asked to choose a plan, the **Free** plan is all you need.

That is it. You now have a place on the internet to store and share projects.

---

4. Configure Git (tell it who you are)

Every commit records who made it. So the first time you use Git on a machine, you tell it your name and email once. Use the **same email** you signed up to GitHub with, so your commits are linked to your account.

Run these two commands, replacing the example values with your own:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

`--global` means "for every project on this computer", so you only do this once, not per project.

It is also worth setting the default name of the starting branch to `main`, which is what GitHub uses. A **branch** is a line of development; you can ignore the deeper meaning for now and just run this so your projects match GitHub's default:

```
git config --global init.defaultBranch main
```

Check your settings

Confirm everything took effect:

```
git config --list
```

You should see your `user.name` and `user.email` in the output. If a value is wrong, just run the matching `git config --global` command again with the correct value; it overwrites the old one.

---

5. Sign in so Git can talk to GitHub (authentication)

Before you can copy a private project or push your changes, GitHub has to know it is really you. GitHub no longer accepts your normal account password from the command line, so you use one of two beginner-friendly methods. Pick one.

Option A (recommended): the GitHub CLI

GitHub publishes a small helper tool called `gh` that handles sign-in for you. Install it from https://cli.github.com, then run:

```
gh auth login
```

Answer the prompts (choose GitHub.com, then HTTPS, then "Login with a web browser"). It opens your browser, you approve once, and from then on Git is signed in. This is the simplest path.

Option B: a Personal Access Token

A **Personal Access Token** (PAT) is a long, random password that you use in place of your real password on the command line.

1. On GitHub, go to **Settings**, then **Developer settings**, then **Personal access tokens**, then **Tokens (classic)**, and select **Generate new token**.
2. Give it a name, pick an expiry, tick the **repo** box, and generate it.
3. Copy the token now and keep it somewhere safe. You will not be able to see it again.

Later, the first time you push or clone a private project, Git will ask for a username and password. Enter your GitHub username, and paste the **token** as the password. Your system will usually remember it after that.

---

6. Clone a repository (copy it to your machine)

Cloning downloads a full copy of a project, with its whole history, onto your computer. You do this once per project.

1. On GitHub, open the project's page. Select the green **Code** button, make sure **HTTPS** is selected, and copy the address shown. It looks like `https://github.com/someone/some-project.git`.
2. In your terminal, move to the folder where you keep your work, then clone. For example:

```
cd ~/projects
git clone https://github.com/someone/some-project.git
```

Git creates a new folder named after the project and fills it with all the files. Move into it:

```
cd some-project
```

You are now standing inside your own copy of the project, ready to work.

---

7. The everyday loop: pull, change, commit, push

This is the rhythm you will repeat every working day. It is five small steps.

Step 1. Pull the latest first

Before you start, bring down anything your teammates changed since you last looked. This keeps your copy current and avoids clashes later.

```
git pull
```

If nothing changed, Git says "Already up to date", which is perfectly fine.

Step 2. Make your change

Edit a file, add a file, or delete one, using your normal editor. Nothing special happens yet; Git is just watching.

Step 3. See and stage what changed

Ask Git what it noticed:

```
git status
```

It lists the files you touched. Now **stage** the ones you want to save. Staging is telling Git "include these in my next snapshot". To stage everything you changed:

```
git add .
```

The `.` means "everything in this folder". You can also name a single file, for example `git add notes.txt`.

Step 4. Commit (save the snapshot)

Record the staged changes as a commit, with a short message that says what you did:

```
git commit -m "Add my notes file"
```

The message in quotes is for your future self and your team, so make it describe the change in plain words. This commit is saved in your **local** history only; GitHub does not have it yet.

Step 5. Push (send it to GitHub)

Send your new commit up to the remote so everyone can see it:

```
git push
```

Refresh the project page on GitHub and you will see your change there. That is the full circle: you pulled the latest, made a change, saved it as a commit, and pushed it back.

---

8. A one-screen reference

Once the ideas above make sense, this short table is all you need day to day.

You want to...	Command
Check Git is installed	`git --version`
Set your name (once per machine)	`git config --global user.name "Your Name"`
Set your email (once per machine)	`git config --global user.email "you@example.com"`
See all your settings	`git config --list`
Copy a project to your machine	`git clone <address>`
Bring down the latest changes	`git pull`
See what you changed	`git status`
Stage your changes	`git add .`
Save a snapshot with a message	`git commit -m "message"`
Send your commits to GitHub	`git push`

---

9. Common first-timer questions

"It asked for a username and password and rejected my password." GitHub does not accept your account password on the command line. Use the GitHub CLI (`gh auth login`) or paste a Personal Access Token as the password. See Section 5.

"My push was rejected because the remote has changes I do not have." Someone pushed while you were working. Run `git pull` to bring their changes in, resolve any overlap it reports, then `git push` again. Pulling before you start (Step 1) makes this rare.

"Did I lose my work?" Almost never. Once you have run `git commit`, that snapshot is safely in your local history even if the push fails. You can push it later.

"What is the difference again between commit and push?" A **commit** saves a snapshot on your own machine. A **push** sends your saved commits up to GitHub. You can make several commits and then push them all at once.

Where this fits

This primer covers just enough Git and GitHub to take part in a real project: install, account, configure, clone, pull, and push. That is the foundation the rest of Day 1 builds on, where a project's history, its commits, and its setup become part of how you write software that a team can rebuild and trust. When you are comfortable with the loop in Section 7, you know enough to begin.